

# NumPy II: broadcasting, and plotting what you compute

Two arrays of different shapes are allowed to work together — under two rules that are entirely mechanical, and that produce the one bug in this course capable of giving you a wrong answer without any error at all. Then Matplotlib, so that from Friday onward we can look at a curve instead of describing one.

Unit 1 · Foundations

80 minutes

Lab 4 · vectorise and plot

Laptops open

Where we are

Broadcasting

The silent bug

Plotting

A curve, end to end

Lab 4

Before Friday

## Where we are

On Friday an array became a thing with a shape, and `.shape` became the first thing you print. Today that pays off twice: once because broadcasting is a rule about shapes and nothing else, and once because the nastiest bug in NumPy is a shape you did not notice.

One number from Lab 3 worth repeating. Most of the room got somewhere between five and eight of the ten shape predictions right. That is the correct score for week two, and it is also why we spend today on the rule that governs what happens when shapes do not match.

# Broadcasting — two rules, and you can predict every result

1

The most powerful and most dangerous feature in NumPy. It is also, mercifully, completely mechanical.

You have already used it. `a + 10` adds a single number to a whole array — the 10 was *broadcast* across every element. The general rule extends that to arrays of different shapes.

## THE ALGORITHM — APPLY IT IN THIS ORDER, EVERY TIME

- 1. Line the shapes up from the right.** If one is shorter, pad it on the *left* with 1s.
- 2. In each column, the two sizes must be equal, or one of them must be 1.** A 1 is stretched to match. Anything else is an error.

## LINE UP FROM THE RIGHT — THREE WORKED CASES

**WORKS** — the 4 matches, the missing row-count is padded to 1 and stretched

|   |        |              |              |                 |                                         |
|---|--------|--------------|--------------|-----------------|-----------------------------------------|
| X | (3, 4) | <div>3</div> | <div>4</div> | → result (3, 4) | one bias per column, added to every row |
| b | (4,)   | <div>1</div> | <div>4</div> |                 |                                         |

**WORKS** — the explicit 1 is stretched across all four columns

|   |        |              |              |                 |                                 |
|---|--------|--------------|--------------|-----------------|---------------------------------|
| X | (3, 4) | <div>3</div> | <div>4</div> | → result (3, 4) | one value per row, added across |
| c | (3, 1) | <div>3</div> | <div>1</div> |                 |                                 |

**FAILS** — 4 and 3 are neither equal nor 1

|   |        |              |              |          |              |              |                     |
|---|--------|--------------|--------------|----------|--------------|--------------|---------------------|
| X | (3, 4) | <div>3</div> | <div>4</div> | d (3,) → | <div>1</div> | <div>3</div> | 4 ≠ 3, neither is 1 |
|---|--------|--------------|--------------|----------|--------------|--------------|---------------------|

| Shape A | Shape B | Result | Why                                                  |
|---------|---------|--------|------------------------------------------------------|
| (3, 4)  | (4,)    | (3, 4) | Padded to (1,4); the 1 stretches to 3.               |
| (3, 4)  | (3, 1)  | (3, 4) | Explicit 1 stretches to 4.                           |
| (3, 1)  | (1, 4)  | (3, 4) | Both stretch. This is the one that surprises people. |
| (3, 4)  | (3,)    | ERROR  | Padded to (1,3). Rightmost column: 4 vs 3. No.       |
| (3,)    | (4,)    | ERROR  | 3 vs 4, neither is 1.                                |

the error you will see more than any other this semester

ERROR

```
np.array([1,2,3]) + np.array([1,2,3,4])
```

```
ValueError: operands could not be broadcast together
with shapes (3,) (4,)
```

Read it as a gift. NumPy has told you both shapes and where they disagree. Nine times in ten the fix is a `.reshape(-1, 1)` or a transpose, and the diagnosis takes ten seconds once you trust the message.

PART 2 · 10 MINUTES · THE IMPORTANT PART OF TODAY

## The bug that does not raise an error

An error is a good day. This is the other kind.

2

You have three predictions and three true values. You want the mean squared error. The predictions came out of one calculation and the targets came from a column of a table — a completely ordinary situation, and one you will be in during session 10.

predict the output. do not scroll.

PYTHON

```
p = np.array([1., 2., 3.])      # predictions      shape (3,)
t = np.array([[1.], [2.], [3.]]) # targets          shape (3, 1)

mse = ((p - t) ** 2).mean()
print(mse)
```

## The predictions are exactly correct. What does it print?

Every prediction matches its target perfectly. Commit to an answer.

It prints **1.3333333333333333**. The predictions are perfect and the loss is not zero.

**Why.** `p` is `(3,)`, padded to `(1, 3)`. `t` is `(3, 1)`. Both have a 1, so both stretch — and the result is a `(3, 3)` matrix of *every prediction minus every target*:

```
[[ 0.  1.  2.]  
 [-1.  0.  1.]  
 [-2. -1.  0.]]
```

The diagonal is the answer you wanted — all zeros. The other six entries are comparisons between unrelated pairs, and the mean is taken over all nine. NumPy did nothing wrong. Row three of the broadcasting table says `(1,3)` and `(3,1)` combine to `(3,3)`, and that is exactly what happened.

**The fix — either one:**

```
((p - t.ravel()) ** 2).mean() # 0.0 - flatten t to (3,)  
((p[:, None] - t) ** 2).mean() # 0.0 - lift p to (3, 1)
```

Both give 0.0. The rule is to make the two shapes agree *before* the arithmetic, not to hope.



### Why I am spending ten minutes on one bug

A model that will not train is usually this. Your loss is a number, so nothing crashes; it goes down, so nothing looks wrong; it plateaus somewhere useless and you spend two days changing the learning rate. In session 16 several of you will hit exactly this, and I would rather you recognise the smell than rediscover it.

#### The habit:

print `.shape` on both sides before any operation you are unsure of. It costs one line and it is the single most effective debugging technique in this entire course.

## Three plots, and no more than three

Matplotlib is enormous. This course uses a small corner of it, and you will use that corner in almost every session from Friday to December.

3

The reason to learn this today rather than later is simple: from session 9 onward, nearly every claim in this course is a claim about a curve. A loss that goes down. A learning rate that is too large. A model that fits the training data and not the test data. Describing those in words is a poor substitute for looking at them, and looking at them costs three lines.

the import, and the only two lines that always appear

PYTHON

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 200)  # 200 evenly spaced numbers from -3 to 3
y = x ** 2

plt.plot(x, y)
plt.show()
```

`np.linspace(a, b, n)` is the function you will reach for constantly:  $n$  evenly spaced values from  $a$  to  $b$ , endpoints included. Compare it with the comprehension you wrote last Tuesday — `[i/10 for i in range(-30, 31)]` — and you have the whole argument for NumPy in one line again.

### Plot one — the line, for anything that changes

this is the plot you will draw most often all term

PYTHON

```
epochs = np.arange(1, 21)
loss    = 2.0 * np.exp(-0.2 * epochs)  # a plausible training curve

plt.plot(epochs, loss, marker="o")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.title("training loss")
plt.grid(True)
plt.show()
```



### An unlabelled axis is a wrong plot

In this course a figure without `xlabel` and `ylabel` is incomplete, and in the midterm it loses marks. This is not fussiness. A loss curve and an accuracy curve look identical and mean opposite things — one going up is good news and the other is a disaster. The label is what carries the meaning.

## Plot two — the scatter, for data points

every dataset in unit 2 gets looked at this way first

PYTHON

```
rng = np.random.default_rng(0)           # seed 0 — everyone gets the same points
hours = rng.uniform(0, 10, 30)
scores = 8 * hours + rng.normal(0, 6, 30)  # a trend, plus noise

plt.scatter(hours, scores)
plt.xlabel("hours studied")
plt.ylabel("exam score")
plt.show()
```

Thirty points, an obvious upward trend, and no line through them yet. In session 10 you will put that line there twice — once solved exactly, once learned by descent — and this scatter is the picture you will put it on.

## Plot three — two things on one pair of axes

the comparison plot: data against model, or train against test

PYTHON

```
plt.scatter(hours, scores, label="data")
plt.plot(hours, 8 * hours, color="red", label="model")
plt.xlabel("hours studied")
plt.ylabel("exam score")
plt.legend()
plt.show()
```

Two calls before one `show` put both on the same axes. Add `label=` to each and call `legend()`, or the reader has to guess which is which. In session 22 this exact plot — two curves, one legend — is how overfitting is diagnosed.



### That is the whole of it

`plot`, `scatter`, labels, `legend`, `show`. There is a great deal more in Matplotlib and you will not need it. When you eventually do — a second panel, a log axis — look it up then, having a reason.

## Everything so far, in six lines

4

An array, a broadcast, and a picture. This is a rehearsal for session 9, where the same six lines draw a loss surface and you find its lowest point by hand.

run this — it is session 9 in miniature

PYTHON

```
w = np.linspace(-2, 4, 200)      # 200 candidate values of a weight
loss = (w - 1.5) ** 2 + 0.5      # the error each one would give

plt.plot(w, loss)
plt.xlabel("weight")
plt.ylabel("loss")
plt.show()

print(w[loss.argmin()])          # 1.497... — the best weight, found by looking
```

`argmin` gives the *position* of the smallest value, which you then use to index back into `w`. Notice what happened: you found the best weight by trying two hundred of them and picking the winner. That works here because there is one weight. In session 13 there are thirteen, and in a real network there are millions — which is precisely why session 9 exists.

### THE LAST OF THE TOOLS

Unit 1 has two sessions left and they are both mathematics rather than programming. You now have everything you need to write it: arrays with shapes, arithmetic without loops, broadcasting, and a way to look at any number you compute. From here the difficulty stops being the language and starts being the ideas — which is the right way round.

## Vectorise five loops, time both, and draw one curve

5

Lab 4 is a separate handout — `CSC4315_Lab04_Vectorise_And_Plot.html`. Five jobs written twice, once as a loop and once as an array operation, timed on your own machine so the speed-up is a number you measured rather than one I claimed. Then one plot, with labels.



### Same rules

Finish in the room, hand up after three minutes, nothing here is a prerequisite for Friday.

## Before Friday

1. **Finish Lab 4**, and write your own timing ratio at the top of the page. Bring that number.
2. **Redraw the loss curve from part 4** with a different minimum — put it at  $w = -0.5$  instead of 1.5 — and confirm `argmin` finds it. Two minutes, and it makes Friday's session concrete before it starts.
3. **If broadcasting still feels like magic**, take the three worked cases from part 1 and do two more of your own on paper: line the shapes up from the right and apply the two rules. This is the one topic where five minutes of paper beats an hour of reading.



### Session 8 — Friday 02 October, 11:30

*The layer is a matrix multiply.* The reveal the whole unit has been building to:  $X @ W + b$  — three symbols you can now read individually — is one complete layer of a neural network. You will compute a  $2 \times 3$  layer by hand on paper, then in one line of NumPy, and check that the two agree digit by digit.